

Acquisition et Nettoyage des Données

Nettoyage approfondi des données

Master 1 Data & IA 2025-2026

Ahmad ALMAKSOUR - Faculté de Gestion, Économie et Sciences

Plan de la séance

01

**Diagnostic
qualité**

Profilage, 4 dimensions

02

**Valeurs
manquantes**

MCAR/MAR/MNAR, imputation

03

Outliers

IQR, z-score, isolation

04

Doublons

Exact, flou, record linkage

05

Transformations

Normalisation, encodage

06

**Pipeline
robuste**

sklearn, reproductibilité

Pourquoi nettoyer ? La règle des 80/20

Une étude IBM estime qu'un data scientist passe 80 % de son temps à nettoyer les données et seulement 20 % à les analyser. Comprendre pourquoi évite d'ignorer cette étape.

Nettoyage et préparation

80 %

Modélisation

20 %

Modèle biaisé

Si les données d'entrée contiennent des valeurs aberrantes ou manquantes mal gérées, le modèle apprend des patterns faux. Garbage in, garbage out.

Décisions erronées

Un dashboard avec des doublons gonfle artificiellement les KPIs. Un dirigeant peut prendre une décision stratégique sur des chiffres faux.

Coût opérationnel

Re-nettoyer des données après mise en prod coûte 10× plus que le faire correctement la première fois (dette technique).

Risque réglementaire

En finance ou santé, livrer un rapport basé sur des données non documentées peut entraîner des sanctions juridiques.

Les 4 dimensions de la qualité des données

Avant de nettoyer, il faut savoir ce qu'on cherche. Quatre dimensions structurent toute analyse de qualité.

Complétude

Toutes les valeurs nécessaires sont-elles présentes ?

- NaN sur surface_reelle_bati
- Champ adresse vide
- Date de mutation manquante

```
isna().sum() / len(df)
```

Validité

Les valeurs respectent-elles les règles métier ?

- Prix négatif
- Code INSEE à 4 chiffres
- Date dans le futur

```
df[df['prix'] < 0]
```

Cohérence

Les valeurs sont-elles cohérentes entre elles ?

- Surface > terrain
- Code postal vs commune
- Total ≠ somme(détails)

```
compare_columns()
```

Unicité

Y a-t-il des doublons ?

- Même transaction répétée
- Doublet flou : Lille vs Lille-Centre

```
df.duplicated().sum()
```

Profilage rapide - Les premières commandes

Avant tout nettoyage, un profilage systématique en 5 minutes révèle 80 % des problèmes. À répéter au début de chaque analyse.

```
import pandas as pd
df = pd.read_parquet('dvf_enrichi.parquet')

# 1. Vue d'ensemble
df.shape          # (n_lignes, n_colonnes)
df.dtypes         # types des colonnes
df.info(memory_usage='deep') # types + non-null + memoire

# 2. Statistiques numeriques
df.describe()
# Reveler : min/max suspects, std=0 (constante), 25%/75% inattendus

# 3. Statistiques categorielles
df.describe(include='object')
# count, unique, top, freq

# 4. Valeurs manquantes
df.isna().sum().sort_values(ascending=False)
df.isna().mean().mul(100).round(1) # en %

# 5. Doublons
df.duplicated().sum()
df.duplicated(subset=['cle']).sum()

# 6. Cardinalite des colonnes
df.nunique().sort_values()
```

df.info()

Voir le memory_usage en deep — révèle les colonnes 'object' qui devraient être catégorielles.

df.describe()

std=0 = colonne constante. min=0 sur une variable strictement positive = anomalie.

df.isna().mean()

> 50 % de NaN sur une colonne ? Souvent inutilisable. Décider de la garder ou drop.

df.nunique()

nunique=1 = colonne inutile. nunique=len(df) sur autre chose qu'un ID = anomalie.

Outils de profilage automatisé

ydata-profiling

Le plus complet. Génère un rapport HTML avec stats, distribution, corrélations, alertes (high cardinality, missing).

```
pip install ydata-profiling
```

```
from ydata_profiling import ProfileReport  
  
ProfileReport(df, title='DVF').to_file('rep.html')
```

sweetviz

Comparaison entre deux datasets ou avant/après nettoyage. Rapport visuel orienté EDA.

```
pip install sweetviz
```

```
import sweetviz as sv  
rep = sv.compare([df1, 'Avant'], [df2, 'Après'])  
rep.show_html('compare.html')
```

missingno

Spécialisé sur les valeurs manquantes. Heatmap, dendrogramme, matrice de corrélation des NaN.

```
pip install missingno
```

```
import missingno as msno  
msno.matrix(df)  
msno.heatmap(df)  
msno.dendrogram(df)
```

Discussion - Qu'est-ce qu'une donnée propre ?

Cas 1

Adresse 'Rue de la Gare' (sans numéro)

Cas 2

Date au format string '15/03/24'

Cas 3

Prix à 0 euro dans DVF

Typologie des valeurs manquantes - MCAR, MAR, MNAR

MCAR Missing Completely At Random	Définition Le manque est totalement aléatoire - indépendant des autres variables.	Exemple DVF <i>Capteur tombé en panne 1 jour. Erreur réseau lors d'un import.</i>	Traitement Toutes les méthodes valides. Drop possible si peu nombreux.
MAR Missing At Random	Définition Le manque dépend d'autres variables observées, pas de la valeur manquante elle-même.	Exemple DVF <i>Surface manquante plus souvent dans les vieux contrats DVF (avant 2018).</i>	Traitement Imputation conditionnelle (par année, par type) recommandée.
MNAR Missing Not At Random	Définition Le manque dépend de la valeur manquante elle-même - biais structurel.	Exemple DVF <i>Salaire manquant : les hauts revenus refusent plus souvent de répondre.</i>	Traitement Très difficile. Modélisation explicite du mécanisme nécessaire.

Stratégie 1 - Suppression (drop)

La méthode la plus simple : supprimer les lignes ou colonnes contenant des NaN. À utiliser uniquement si on perd peu d'information.

```
# Suppression de lignes
df.dropna() # toute ligne avec un NaN
df.dropna(subset=['valeur_fonciere']) # uniquement si NaN sur cette colonne

# Suppression de colonnes
df.dropna(axis=1) # toute colonne avec au moins un NaN

# Approche pratique : drop des colonnes trop incomplètes
seuil = 0.5 # 50 %
cols_a_drop = df.columns[df.isna().mean() > seuil]
print(f'Colonnes droppees : {cols_a_drop.tolist()}')
df = df.drop(columns=cols_a_drop)

# Verifier l'impact avant de valider
n_avant = len(df)
df_clean = df.dropna()
print(f'Lignes : {n_avant} -> {len(df_clean)}  
{(len(df_clean)/n_avant*100:.1f)}%')
```

Drop ligne

Quand : peu de lignes affectées (< 5 %), MCAR confirmé, pas d'analyse longitudinale.

Drop colonne

Quand : > 50 % de NaN, colonne non critique, pas d'imputation fiable possible.

Ne PAS drop

Petit dataset (< 1000 lignes), MAR/MNAR (biais), donnée critique métier.

Stratégie 2 — Imputation simple (mean, median, mode)

Remplacer les NaN par une valeur calculée : moyenne, médiane, mode ou constante. Rapide, mais réduit la variance et peut introduire un biais.

```
from sklearn.impute import SimpleImputer
import pandas as pd

# Option 1 : pandas direct
df['surface_reelle_bati'].fillna(df['surface_reelle_bati'].median(), inplace=True)
df['nom_commune'].fillna('Inconnu', inplace=True)

# Option 2 : sklearn (recommande pour pipelines reproductibles)
imp_num = SimpleImputer(strategy='median')
df[['surface', 'prix']] = imp_num.fit_transform(df[['surface', 'prix']])

imp_cat = SimpleImputer(strategy='most_frequent')
df[['type_local']] = imp_cat.fit_transform(df[['type_local']])

# Imputation par valeur fixe
imp_zero = SimpleImputer(strategy='constant', fill_value=0)

# Imputation conditionnelle (groupby)
# Imputer par la mediane DU TYPE DE BIEN, pas la mediane globale
df['surface'] = df.groupby('type_local')['surface'].transform(
    lambda x: x.fillna(x.median())
)
```

Quelle stratégie ?

Numérique
symétrique

mean

Salaires non skewed

Numérique
asymétrique

median

Prix immo (queue à droite)

Catégoriel

most_frequent

type_local

Métier

constant

'Inconnu' explicite

Stratégie 4 - Imputation par modèle (KNN, Iterative)

Pour les NaN sur plusieurs colonnes corrélées, un modèle prédictif donne de meilleurs résultats que la médiane globale.

```
from sklearn.impute import KNNImputer, IterativeImputer
# IterativeImputer est experimental dans sklearn
from sklearn.experimental import enable_iterative_imputer

# === KNN Imputer ===
# Pour chaque NaN : trouver les K voisins les plus proches dans
# l'espace des autres variables, prendre la moyenne de leurs valeurs.
knn = KNNImputer(n_neighbors=5, weights='distance')
cols_num = ['surface', 'prix', 'pop_commune', 'lat', 'lon']
df[cols_num] = knn.fit_transform(df[cols_num])

# === Iterative Imputer (MICE) ===
# Modelise chaque colonne comme une fonction des autres,
# itere jusqu'a convergence.
# Plus precis mais plus lent.
from sklearn.linear_model import BayesianRidge
ii = IterativeImputer(estimator=BayesianRidge(), max_iter=10, random_state=42)
df[cols_num] = ii.fit_transform(df[cols_num])
```

Méthode	Temps	Qualité	Cas d'usage
Drop	Instantané	N/A	< 5 % de NaN, MCAR
Médiane	Instantané	Médiocre réduit variance	Prototypage rapide
Conditionnelle (groupby)	Rapide	Bonne	MAR avec groupes connus
KNN	Lent	Très bonne	Variables corrélées
Iterative (MICE)	Très lent	Excellente	Datasets critiques

Exercice live — Diagnostiquer et imputer

Pour ce DataFrame DVF, choisissez la meilleure stratégie pour chaque colonne et expliquez pourquoi.

```
df.isna().mean().mul(100).round(1)
```

```
# Resultat :
```

```
valeur_fonciere    0.0  
date_mutation      0.2  
type_local         0.0  
surface_reelle_bati 3.5  
nb_pieces          8.2  
surface_terrain    67.4  
adresse_numero     18.6  
adresse_nom_voie   2.1  
nom_commune        0.0  
lat               75.3  
lon               75.3
```

surface_reelle_bati 3.5 % NaN

nb_pieces 8.2 % NaN

surface_terrain 67.4 % NaN

adresse_numero 18.6 % NaN

Outlier - Définition statistique vs métier

Un outlier est une observation qui dévie significativement du reste des données. Mais quoi qu'en dise les statistiques, c'est l'expertise métier qui décide.

Définition statistique

Une valeur est outlier si elle s'écarte d'une distribution attendue selon un seuil (3σ , $1.5 \times IQR$, ou un quantile extrême).

Approche purement mathématique : la donnée est suspecte parce qu'elle est rare.

Définition métier

Une valeur est outlier si elle est incohérente avec la réalité du domaine (prix immobilier > 50 M€, surface > 10 000 m²).

Approche pragmatique : la donnée est suspecte parce qu'elle viole une règle métier connue.

Un outlier est-il une erreur, ou un signal métier ?

Erreur de saisie

Ex : surface=655555 m² (digit en trop). À corriger ou supprimer.

Cas atypique légitime

Ex : appartement de luxe à 5M€. À conserver mais isoler dans l'analyse.

Phénomène rare important

Ex : crash boursier, COVID. Le supprimer cache un signal essentiel.

Doublons flous — Quand l'égalité stricte ne suffit plus

Dans la vraie vie, des entités identiques apparaissent avec des orthographes différentes. La déduplication floue mesure la similarité plutôt que l'égalité.

Exemples de doublons que duplicated() ne détecte PAS

Champ 1	Champ 2	Cause du doublon
Villeneuve-d'Ascq	Villeneuve d Ascq	Apostrophe + tiret
Lille	LILLE	Casse + espace final
St-Saulve	Saint-Saulve	Abréviation
Marcq-en-Barœul	Marcq en Baroeul	Caractère spécial œ
59650	59 650	Espace dans code postal

Stratégies de déduplication floue

Normalisation

unidecode + upper + remove punct. Rapide, déterministe.
Couvre 80 % des cas dans les noms français.

Distance d'édition

Levenshtein, Jaro-Winkler. Mesure le nombre d'opérations pour passer d'une chaîne à l'autre.

Phonétique

Soundex, Metaphone. Compare la prononciation. Utile pour les noms propres.

RapidFuzz - Déduplication floue en pratique

RapidFuzz est la bibliothèque Python de référence pour la similarité de chaînes. Plus rapide que fuzzywuzzy, même API. Utilise une implémentation C++.

```
from rapidfuzz import fuzz, process

# === Score de similarité entre 2 chaînes (0-100) ===
fuzz.ratio('Villeneuve-d Ascq', 'Villeneuve d Ascq')      # 97
fuzz.ratio('Lille', 'LILLE')                               # 0 (case-sensitive)
fuzz.ratio('lille', 'lille')                               # 100

# === Variantes ===
fuzz.partial_ratio('St Pol', 'Saint Pol sur Mer')         # 100 (substring)
fuzz.token_sort_ratio('Pol Saint', 'Saint Pol')           # 100 (tokens)
fuzz.token_set_ratio('Saint Pol Mer', 'St Pol Mer')       # ~75

# === Recherche du meilleur match dans une liste ===
candidats = ['Lille', 'Roubaix', 'Tourcoing', 'Wattrelos']
cible = 'Lile'      # erreur de saisie
best = process.extractOne(cible, candidats, scorer=fuzz.ratio)
# ('Lille', 88, 0) -> (match, score, index)

# === Application sur DataFrame ===
def trouver_match(nom: str, ref: list, seuil: int = 85):
    match = process.extractOne(nom, ref, scorer=fuzz.ratio)
    return match[0] if match and match[1] >= seuil else None

df['nom_canonique'] = df['nom_commune'].apply(
    lambda x: trouver_match(x, liste_communes_officielles)
)
print(f'Communes résolues : {df.nom_canonique.notna().sum()} / {len(df)}')
```

Pipeline de déduplication - De brut à canonique

En production, la déduplication n'est jamais une seule étape. Elle combine normalisation, matching flou et résolution.



Encodage des variables catégorielles

Les modèles ML ne comprennent pas les chaînes de caractères. Quatre techniques pour transformer une catégorie en nombre.

Label Encoding

0, 1, 2, 3...

```
Sklearn LabelEncoder
```

Implique un ordre artificiel. À éviter sauf pour les catégories ORDINALES (Faible, Moyen, Fort).

One-Hot

Une colonne binaire par catégorie

```
pd.get_dummies(df, columns=['type_local'])
```

Pas d'ordre implicite. Explose le nombre de colonnes si haute cardinalité (> 50 valeurs).

Ordinal Encoding

Ordre métier explicite

```
OrdinalEncoder(categories=[[ 'Bas', 'Moyen', 'Haut ' ]])
```

Préserve l'ordre métier. À utiliser quand l'ordre fait sens (gamme, niveau, taille).

Parsing et features de dates

Une date n'est pas qu'un nombre - elle contient année, mois, jour de la semaine, saison. Bien parser ouvre des features ML très utiles.

```
import pandas as pd

# 1. Parsing avec format explicite (recommande)
df['date_mutation'] = pd.to_datetime(
    df['date_mutation'],
    format='%Y-%m-%d',
    errors='coerce' # NaT si parsing impossible
)

# Plusieurs formats melanges : essayer plusieurs patterns
def parse_date_robust(s):
    for fmt in ['%Y-%m-%d', '%d/%m/%Y', '%Y%m%d']:
        try: return pd.to_datetime(s, format=fmt)
        except: continue
    return pd.NaT

# 2. Extraction de features
df['annee'] = df['date_mutation'].dt.year
df['mois'] = df['date_mutation'].dt.month
df['jour_sem'] = df['date_mutation'].dt.dayofweek # 0=lundi
df['trimestre'] = df['date_mutation'].dt.quarter
df['est_weekend'] = df['jour_sem'].isin([5, 6])
df['saison'] = df['mois'].map({12:'hiver',1:'hiver',2:'hiver',
                               3:'printemps',4:'printemps',5:'printemps',
                               6:'ete',7:'ete',8:'ete',
                               9:'automne',10:'automne',11:'automne'})
```

Format ambigu

'01/02/2024' = 1er fév ou 2 janv ? Toujours spécifier dayfirst=True ou format explicite.

Fuseaux horaires

Distinguer naive (sans tz) et aware (avec tz). Toujours convertir en UTC pour stockage.

Heure d'été

Le 27 octobre 2h existe deux fois. Source de doublons surnois.

Regex pour extraire et nettoyer du texte

Les expressions régulières (regex) sont l'outil universel pour extraire des patterns dans du texte non structuré. Indispensable pour parser des adresses, codes, numéros.

```
import re
import pandas as pd

# === Patterns Python re ===
# \d : un chiffre      \D : pas un chiffre
# \w : alphanum        \s : espace
# + : 1 ou plus        * : 0 ou plus
# ? : 0 ou 1           ^ : debut $ : fin
# [] : ensemble        () : groupe de capture
# {n} : exactement n   {n,m} : entre n et m

# === Sur Series pandas avec .str ===
# Extraire le code postal d'une adresse
df['cp'] = df['adresse'].str.extract(r'(\d{5})')

# Extraire le numero de rue
df['num'] = df['adresse'].str.extract(r'^(\d+)\s')

# Tester si une adresse contient 'rue'
df['contient_rue'] = df['adresse'].str.contains(r'\b[Rr]ue\b', regex=True)

# Remplacer plusieurs patterns
df['adresse_clean'] = (df['adresse']
    .str.replace(r'\s+', ' ', regex=True)      # espaces multiples
    .str.replace(r'^\s+|\s+$', '', regex=True) # trim
)

# Extraire plusieurs groupes
# 'Lille (59000)' -> nom='Lille', cp='59000'
df[['nom', 'cp']] = df['ville'].str.extract(r'([A-Za-z\ - ]+)\s*\((\d{5})\)')
```

Pipeline sklearn - Industrialiser le nettoyage

Plutôt qu'enchaîner manuellement les transformations, sklearn Pipeline les compose en un objet unique. Avantage : traitement train/test cohérent, sérialisation, fit_transform().

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, RobustScaler

# === Définir les transformations par type de colonne ===
preproc_num = Pipeline(steps=[
    ('imputer', KNNImputer(n_neighbors=5)),
    ('scaler', RobustScaler()),
])

preproc_cat = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
])

# === Combiner par colonne ===
cols_num = ['valeur_fonciere', 'surface_reelle_bati', 'pop_commune']
cols_cat = ['type_local', 'arrondissement']

preprocesseur = ColumnTransformer(transformers=[
    ('num', preproc_num, cols_num),
    ('cat', preproc_cat, cols_cat),
])

# === Application ===
X_clean = preprocesseur.fit_transform(df_train) # fit + transform sur train
X_test_clean = preprocesseur.transform(df_test) # transform seul sur test

# IMPORTANT : ne JAMAIS faire fit_transform sur le test
# Cela injecte de l'information du test dans le train (data leakage)
```

Reproductibilité — Versionner et sauvegarder

Un pipeline de nettoyage doit être reproductible : même données + même code = même résultat. Cela impose de versionner ET les données ET les transformations.

Sérialiser le pipeline

Sauvegarder l'objet sklearn fitté pour le réutiliser sans re-entraîner.

```
import joblib
joblib.dump(preprocesseur, 'preproc_v1.pkl')
p = joblib.load('preproc_v1.pkl')
```

Logger les paramètres

Dans un fichier .yaml ou .json, garder trace de tous les seuils utilisés (IQR k, contamination, NaN strategy).

```
params = dict(
    iqr_k=1.5,
    contamination=0.05,
    imputer='knn',
)
```

Hasher les datasets

Calculer un hash MD5/SHA des fichiers d'entrée. Détecte si les données ont changé.

```
import hashlib
h = hashlib.md5(open('dvf.parquet', 'rb').read())
.hexdigest()
print(f'Hash : {h}')
```

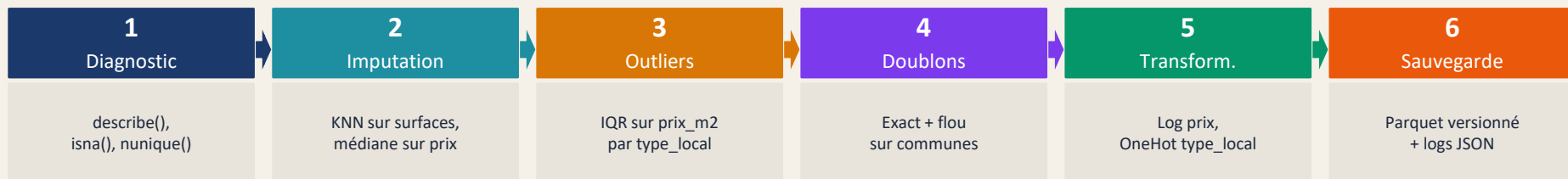
Tracer les transformations

Enregistrer chaque étape dans un fichier log : nb lignes avant/après, NaN avant/après, hash.

```
log = {
    'step': 'imputation',
    'rows_before': 50000,
    'rows_after': 49850,
    'nan_dropped': 150
}
json.dump(log, open('audit.json', 'w'))
```

Pipeline complet DVF — De brut à propre

Synthèse : voici l'enchaînement complet appliqué au pipeline fil rouge depuis la séance 1.



Discussion finale - 3 dilemmes du nettoyage

Deux situations qui n'ont pas de réponse unique. Réfléchissons aux compromis.

1

Imputer ou drop ?

Vous avez 50 000 lignes et 30 % de NaN sur la colonne 'nb_pieces'.

2

Déduplication automatique ou manuelle ?

RapidFuzz détecte 'Saint-Saulve' = 'St-Saulve' à 92 %. Mais aussi 'Saint-Pol' = 'Saint-Pol-sur-Mer' à 88 % qui sont en fait deux communes distinctes. Automatiser : risque de faux positifs. Manuel : ne passe pas à l'échelle.

Anti-patterns — Ce qu'il ne faut pas faire

Erreurs classiques rencontrées en mission. Si vous les évitez, vous êtes déjà au-dessus de la moyenne.



Drop sans regarder

`df.dropna()` sans avoir profilé. Vous perdez 30 % des lignes silencieusement.



Imputation par 0

Remplacer NaN par 0 sur des prix, populations, surfaces. Crée des aberrations statistiques.



fit sur train+test

`fit_transform()` sur l'ensemble du DataFrame puis split. Data leakage massif, modèle trompeusement bon.



Outliers supprimés silencieusement

`df = df[df['prix'] < 1e6]` sans logger. Le métier ne sait pas pourquoi son bien à 1.5M€ a disparu.



Pipeline non versionné

Le pickle écrase l'ancien sans numéro de version. Impossible de revenir en arrière.